



ADVANCED JAVASCRIPT · 6-DAY COURSE

# Day 5: Types & Error Handling

Definitions + two real-world examples for every concept

● ● ● day5.js

```
// Run with: node day5.js  
topics = ['coercion', 'try/catch', 'custom errors'];
```

# Today's 5 concepts

01

Type Coercion & Equality — == vs === and truthy/falsy

02

typeof, instanceof & Type-Checking Patterns

03

try/catch/finally & Custom Error Classes

04

Error Handling in Async Code

05

Global Error Handlers & Defensive Patterns

# Type Coercion & Equality

### DEFINITION

JavaScript automatically converts values between types in many operations. The `==` operator coerces both sides to a common type before comparing; `===` compares type AND value with no coercion. Every value is also either 'truthy' or 'falsy' in a boolean context.

### KEY POINTS

- Only 6 falsy values: `false`, `0`, `''`, `null`, `undefined`, `NaN` — rest are truthy
- `==` coerces types (`1 == '1'` is true); `===` never does (`1 === '1'` is false)
- Default to `===` almost everywhere — it's predictable, `==` often surprises
- Coercion edge cases exist (`[] == false` is true) — another reason to avoid `==`

### EXAMPLE 1

```
day5.js

1 == '1';           // true  - coerced to compare
1 === '1';          // false - different types

null == undefined;  // true
null === undefined; // false

// falsy values - memorize these six:
Boolean(false);     // false
Boolean(0);          // false
Boolean('');         // false
Boolean(null);       // false
Boolean(NaN);        // false
Boolean([]);         // true  ← surprising!
```

## REAL-WORLD EXAMPLE

# Example 2 — The 'Falsy Zero' Bug in a Quantity Field

A cart form checks whether a quantity was entered using a plain falsy check. It breaks the moment someone legitimately sets quantity to 0 — a genuinely common production bug caused by mixing up 'empty' with 'falsy'.

```
quantity-check.js

// ✗ 0 is falsy — this treats a valid '0' as 'missing'
function validateQuantity(qty) {
  if (!qty) {
    return 'Quantity is required'; // fires even for qty = 0
  }
  return null;
}

// ✓ explicitly check for missing, not merely falsy
function validateQuantityFixed(qty) {
  if (qty === undefined || qty === null || qty === '') {
    return 'Quantity is required';
  }
  return null; // qty = 0 now passes correctly
}
```

**Why it matters:** Confusing 'falsy' with 'missing' is one of the single most common real bugs — it silently breaks any field where 0 or an empty array is a valid value.

# typeof, instanceof & Type-Checking

### DEFINITION

typeof returns a string naming a value's primitive type. instanceof checks whether an object's prototype chain includes a given constructor. Together with Array.isArray() and explicit null checks, they're how JS code figures out what it's working with at runtime.

### KEY POINTS

- typeof null === 'object' — a famous, long-standing JS quirk
- typeof [] === 'object' too — use Array.isArray() to detect arrays
- instanceof walks the prototype chain — great for custom classes/errors
- Robust type-checks usually combine several of these checks together

### EXAMPLE 1

```
day5.js

typeof 42;           // 'number'
typeof 'hi';         // 'string'
typeof undefined;    // 'undefined'
typeof null;         // 'object' ← the famous quirk
typeof [];           // 'object' ← arrays too

Array.isArray([]);   // true — the real array check

class ApiError extends Error {}
const err = new ApiError('failed');
err instanceof Error;    // true
err instanceof ApiError; // true
```

## REAL-WORLD EXAMPLE

# Example 2 — Safely Narrowing an Unknown API Payload

A generic API client receives JSON of unknown shape and must safely check its type before using it — exactly how real client libraries guard against malformed or unexpected server responses before handing data to the rest of the app.

```
safe-parse.js

function parseApiResponse(data) {
  if (data instanceof Error) {
    throw data; // already an Error object - rethrow
  }
  if (Array.isArray(data)) {
    return data.map(parseApiResponse); // recurse into a list
  }
  if (data === null || typeof data !== 'object') {
    return data; // primitive - nothing to unwrap
  }
  return { ...data, receivedAt: new Date() };
}
```

**Why it matters:** Real API clients can't trust server data blindly — this layered `typeof/instanceof/Array.isArray` check is standard defensive parsing before data reaches the UI.

# try/catch/finally & Custom Error Classes

## DEFINITION

try/catch/finally lets you run code that might throw, handle the failure, and always run cleanup afterward. Custom Error subclasses attach extra context (like a status code) and let you distinguish error types cleanly with instanceof.

## KEY POINTS

- try runs code; catch handles anything thrown; finally always runs
- class MyError extends Error — custom name, extra fields, instanceof checks
- Branch inside catch using instanceof to handle different errors differently
- Never swallow an error silently — always log it or rethrow it

## EXAMPLE 1

```
day5.js

class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}

try {
  throw new ValidationError('email', 'Invalid email');
} catch (err) {
  if (err instanceof ValidationError) {
    highlightField(err.field);
  }
} finally {
  hideLoadingSpinner(); // always runs
}
```

## REAL-WORLD EXAMPLE

# Example 2 — Distinct Errors Driving Real UI Branching

An API layer throws different Error subclasses for different failure types. The calling UI code uses instanceof to react correctly: redirect to login on auth failure, show a 404 page on missing data, or a generic error otherwise.

```
api-errors.js

class UnauthorizedError extends Error {}
class NotFoundError extends Error {}

async function loadOrderPage(id) {
  try {
    const order = await fetchOrder(id); // throws typed errors
    renderOrder(order);
  } catch (err) {
    if (err instanceof UnauthorizedError) {
      redirectToLogin();
    } else if (err instanceof NotFoundError) {
      render404Page();
    } else {
      renderGenericError();
    }
  }
}
```

**Why it matters:** *Typed errors plus instanceof branching is exactly how production apps show a login redirect vs a 404 page vs a generic error, from one catch block.*



# Error Handling in Async Code

### DEFINITION

Errors in Promises don't behave like synchronous throws — a rejected Promise with nothing watching it becomes an 'unhandled rejection' instead of crashing loudly. try/catch around await, and .catch() on every promise, are both needed for solid async error handling.

### KEY POINTS

- A rejected Promise with no .catch() becomes an 'unhandledRejection'
- try/catch around 'await' works exactly like synchronous try/catch
- An error thrown inside .then() propagates to the next .catch()
- Always handle every promise — both awaited AND fire-and-forget ones

### EXAMPLE 1

```
day5.js

async function getUserSafely(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) throw new Error('Fetch failed');
    return await res.json();
  } catch (err) {
    logError(err);
    return null; // caller gets null, not a crash
  }
}

// ✗ fire-and-forget with NO .catch — silent unhandled rejection
getUserSafely('bad-id'); // safe here — try/catch inside handles it
```

## REAL-WORLD EXAMPLE

# Example 2 — Fire-and-Forget Analytics That Never Crash the App

A page logs an analytics event after load without awaiting it, so the UI isn't blocked. If that network call fails, it must NOT surface as an unhandled rejection or break the page — handled defensively with its own `.catch()`.

```
analytics.js

function logPageView(page) {
  // ✗ un-awaited promise with no .catch — risky
  // fetch('/api/track', { method: 'POST', body: page });

  // ✔ defensive fire-and-forget — errors handled locally
  fetch('/api/track', { method: 'POST', body: page })
    .catch((err) => {
      console.warn('Analytics failed silently:', err);
      // intentionally NOT rethrown — analytics is non-critical
    });
}

logPageView('/checkout'); // runs, page keeps working either way
```

**Why it matters:** Non-critical background calls (analytics, telemetry, prefetching) must fail quietly — this pattern keeps a flaky third-party endpoint from ever breaking the page.

# Global Error Handlers & Defensive Patterns

### DEFINITION

Beyond local try/catch, real apps register global handlers as a last-resort safety net: `window 'error'/'unhandledrejection'` events in browsers, or `process.on('uncaughtException'/'unhandledRejection')` in Node — to log and gracefully recover from anything that slips past local handling.

### KEY POINTS

- Global handlers are a SAFETY NET, not a substitute for local try/catch
- Browser: `window.addEventListener('error' / 'unhandledrejection', ...)`
- Node: `process.on('uncaughtException' / 'unhandledRejection', ...)`
- Pair with a fallback UI so users see something graceful, not a blank page

### EXAMPLE 1

```
day5.js

// browser: catch anything that slipped past local handling
window.addEventListener('error', (event) => {
  reportToMonitoring(event.error);
});

window.addEventListener('unhandledrejection', (event) => {
  reportToMonitoring(event.reason);
  event.preventDefault(); // stop the noisy console
                             warning
});

// Node.js equivalent
process.on('uncaughtException', (err) =>
  reportToMonitoring(err));
```

## REAL-WORLD EXAMPLE

# Example 2 — An Error Boundary Backed by a Global Safety Net

A UI component catches its own rendering errors and shows a friendly fallback instead of a blank screen. Global window handlers catch anything outside that render cycle — like a broken click handler — so nothing ever fails completely silently.

```
error-boundary.js

class ErrorBoundary extends Component {
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  componentDidCatch(err) { reportToMonitoring(err); }
  render() {
    return this.state.hasError
      ? <FriendlyErrorScreen />
      : this.props.children;
  }
}

// global net — catches errors OUTSIDE React's render cycle too
window.addEventListener('error', (e) => reportToMonitoring(e.error));
```

**Why it matters:** *Layered defense — component-level boundaries plus a global net — is exactly how production apps avoid ever showing a fully blank, broken page to a user.*

## RECAP

# Day 5 in one glance

- ✓ Coercion: default to `===`, know the 6 falsy values, watch for `'0'`
- ✓ `typeof/instanceof`: combine checks — `null` and arrays need special care
- ✓ `try/catch/finally`: custom `Error` classes make `instanceof` branching clean
- ✓ Async errors: `try/catch` around `await`, `.catch()` on every fire-and-forget call
- ✓ Global handlers: a safety net for anything that slips past local handling

*Practice: write a custom `NetworkError` class, and fix a form's validation bug caused by a falsy-zero check.*